06 · Message hooks

Siddharth M. Bhatia · siddharth@sidmb.com
July 1, 2026

Contents

Interface 1
Attach hooks 2
An LLM-judge hook 2

A `MessageHook` is middleware that inspects every freshly generated message **before it is committed** to the transcript, and may **approve**, **deny** (drop the turn), or **edit** (replace it). Hooks are runtime policy — they live on the live `Conversation` (`conversation.message_hooks`) and are **not** serialized in the template. This is the seam for a safety filter, format enforcer, or LLM-judge that vets/rewrites turns.

Interface

```
from interlens import MessageHook, MessageHookResult

class NoProfanity(MessageHook):
    BANNED = {"heck", "darn"}
    def review(self, message, conversation) → MessageHookResult:
        if any(w in message.content.lower() for w in self.BANNED):
            return MessageHookResult.deny()                 # drop the
     turn entirely
        return MessageHookResult.approve()                  # let it
     through unchanged

class EnforceBrevity(MessageHook):
    def review(self, message, conversation):
        if len(message.content) > 500:
            from interlens import Message
            trimmed = Message(author=message.author,
     content=message.content[:500] + " …",
                              metadata={**message.metadata, "trimmed":
     True})
            return MessageHookResult.edit(trimmed)          # substitute
     a replacement
        return MessageHookResult.approve()
```

`MessageHookResult`
factory helpers: `.approve()`, `.deny()`, `.edit(replacement_message)`.

Attach hooks

```
conv.message_hooks = [NoProfanity(), EnforceBrevity()]     # applied in
     order
result = conv.step(alice)                                  # returns None
     if a hook DENIED the turn
```

Hooks run inside both `step` and `run`. A denied turn commits nothing and `step` returns `None`; `run` simply moves on (and a denied message is not checked against stop conditions).

An LLM-judge hook

Because `review` receives the live `conversation`, a hook can call an API model (or a local one via `conv.sample`) to score the turn:

```
from interlens import MessageHook, MessageHookResult, APIParticipant

class JudgeHook(MessageHook):
    def __init__(self):
        self.judge = APIParticipant(name="judge", model_id="claude-
     sonnet-5",
                                    system_prompt="Reply only 'OK' or
     'BLOCK'. BLOCK if the message is off-topic.")
    def review(self, message, conversation):
        verdict = self.judge.generate([{"role": "user", "content":
     message.content}]).content
        return MessageHookResult.deny() if "BLOCK" in verdict else
     MessageHookResult.approve()

conv.message_hooks = [JudgeHook()]
```

Next: interpretability.

© 2026 Siddharth M. Bhatia. All rights reserved. Interlens is licensed under the GNU Affero General Public License v3.0.